Advanced Questions: Functions, Function Calls and Recursion

Functions (8 Difficult Questions)

1. Explain the scope and lifetime of local, global, static, and register variables with suitable examples.

2. Can a function return multiple values in C? Discuss different techniques and their trade-offs.

3. Analyze the impact of function prototypes on type checking and separate compilation.

4. Explain how static variables inside functions retain state across multiple calls.

5. Differentiate between library functions and user-defined functions with practical applications.

6. What problems can occur when a function is called before its declaration? Explain in detail.

7. Design a function-based solution for a large application and explain how modularity is achieved.

8. Trace and predict the output of a program containing multiple nested function calls.

Function Calls (8 Difficult Questions)

1. Explain the complete sequence of events that occur during a function call at run time.
2. Describe the structure and role of an activation record (stack frame).
3. Illustrate parameter passing using memory diagrams for both call by value and call by reference.
4. Explain how the run-time stack manages nested function calls and returns.
5. Analyze the overhead associated with function calls and methods to reduce it.
6. Explain return address management during deeply nested function calls.
7. Trace stack contents for a program involving three levels of nested function calls.
8. Predict the output and memory changes in a program that modifies variables using pointers.

Recursion (8 Difficult Questions)

1. Differentiate between direct recursion, indirect recursion, and tail recursion with examples.

2. Explain the recursion stack and trace all stack frames for factorial(5).

3. Draw the complete recursion tree for Fibonacci(6) and determine its time complexity.

4. Why is a base condition essential? Explain the consequences of omitting it.

5. Compare recursion and iteration with respect to memory usage, execution speed, and maintainability.

6. Write and explain a recursive algorithm to reverse a number and analyze its complexity.

7. Explain stack overflow in recursion and discuss techniques to avoid it.

8. Given a recursive function $F(n)=F(n-1)+F(n-2)$, draw the recursion tree for F(5) and suggest an optimized solution.